

# 单 Path0 读取估算与自然错开：独立说明 v3

最终单位：1 I/O = 1 KV block = 128 token

2026-09-06

## 最终单位：一块 KV，恰好一条 I/O

独立文件是 glm\_path0\_deadline.py，只依赖 Python 标准库。它输出**本批从当前快照到最后一块 I/O 在 SSD 完成的预计耗时**，不是 NPU 计算时间，也不是实测值。

用户已经明确最终定义：

$$\begin{aligned} 1 \text{ I/O} &= 1 \text{ KV block} = 128 \text{ token 的 KV,} \\ S &= 128 \times 1408 = 180224 \text{ byte} \\ &= \mathbf{176 \text{ KiB.}} \end{aligned}$$

因此，**输入 10 个 KV block，就读取 10 条 I/O**；输入 129 个 KV block，就读取 129 条 I/O。没有再除以 128，没有把多个 block 聚成一条，也没有补齐一条大 I/O 的规则。

1408 byte/token/layer 沿用项目数据格式：data 的 84 条画像中，每层 KV 字节数除以 SSD 前缀 token 数均为 1408（浮点误差内）；合成画像脚本也使用这个常数。**这里算的是每层 KV；该字节数不是对所有 GLM5.1 精度、压缩方式或分片配置的普遍保证。**

## 函数中哪些量固定，哪些由你输入？

固定的是每个 block 的 token 数、每 token 每层的 KV 字节数，以及“一 block — I/O”的对应关系。函数不再提供这些项目的配置参数，调用时只输入本批大小、时间预算、前方队列数和有效带宽；必要时加上在途剩余字节。

当前 sim.py 的 BLOCK\_SIZE=128 也表示 **128 token/I/O**，所以其**完整** I/O 与本工具同为 176 KiB。但有两个边界需要记住：

- 原仿真允许最后一条不足 128 token，本工具假定每条都为完整 block。
- 原仿真逐条提交，其他请求可能交错；本工具假设本批立即完整排入 FIFO。

例如旧反例 NPU0 的 855-token 前缀为 6 条完整 I/O 加 1 条 87-token 尾部。若只填 7 个完整 block，工具按 896 token 计算，会多算 41 token 的读取量。**粒度相同，不代表任意 trace 都能只靠 I/O 条数精确复现。**

本 v3 采用以上最终确认的单位；v1、v2 仅作为旧版记录保留，调用方法请以本版为准。

## 四个主输入，怎样算出读取耗时

| 输入                     | 本版含义                                |
|------------------------|-------------------------------------|
| kv_block_count         | 本批需要的 KV block 总数，也就是本批 I/O 条数      |
| deadline_ms            | 从当前快照起，还剩多少毫秒可供读取                   |
| queued_io_count        | 前方完整等待的 I/O 条数，每条 176 KiB；不含本批与在途一条 |
| bandwidth_gib_s        | 有效盘带宽，二进制 GiB/s，默认 40               |
| active_remaining_bytes | 可选：正在服务的一条还剩多少字节，默认 0，即没有额外在途剩余工作   |

本批立即完整进入同一个 Path0；单串行 SSD、严格 FIFO、固定带宽，后续请求不插到本批之前或中间。令  $K$  为本批 block 数， $N$  为本批 I/O 条数， $Q$  为前方完整等待条数， $A$  为在途剩余字节， $S = 180224$  byte， $B$  为 byte/s：

$$N = K, \quad B = \text{bandwidth\_gib\_s} \times 2^{30},$$

$$T_{\text{queue}} = \frac{QS + A}{B} \times 1000, \quad T_{\text{own}} = \frac{NS}{B} \times 1000,$$

$$T_{\text{read}} = T_{\text{queue}} + T_{\text{own}}, \quad \text{slack} = \text{deadline\_ms} - T_{\text{read}}.$$

以上时间均为毫秒。slack 非负即满足；正值是提前时间，负值的绝对值是欠缺时间。**deadline 不改变 FIFO 服务速度**。空批次  $K = 0$  直接返回耗时 0，不等旧队列；这是业务语义，不是输入防御检查。

### 手算例子：前方 100 条，本批 10 条

取  $K = 10$ 、 $Q = 100$ 、 $A = 0$ 、带宽 40 GiB/s、deadline 为 0.5 ms。前方和本批的每条都为 176 KiB：

| 计算项               | 结果                           |
|-------------------|------------------------------|
| 每条 I/O 服务时间       | 0.0041961669921875 ms        |
| 等前面 100 条读完       | 0.41961669921875 ms          |
| 自己 10 条的服务        | 0.041961669921875 ms         |
| 预计读取耗时            | <b>0.461578369140625 ms</b>  |
| 距 0.5 ms deadline | <b>提前约 0.038421631 ms，满足</b> |

精确提前量为 0.038421630859375 ms。deadline 若改为 0.4 ms，耗时仍为 0.461578369140625 ms，只是欠缺 0.061578369140625 ms。

这个例子只算已经确定的 FIFO 顺序，不会模拟未知的未来交错提交。需要预测 NPU 何时恢复计算，还必须考虑材料到 HBM 的时间，以及当前计算何时结束。

## 精简接口、调用方法与适用边界

函数保留四个主输入和一个可选的在途剩余字节，直接返回普通字典。没有 dataclass，也没有 bool、NaN、溢出、负值或多格式的防御性检查。**调用方负责提供有效输入**：数量为非负整数、带宽为正、在途剩余字节符合实际的一条 I/O；deadline 使用当前快照的剩余预算。

```
from glm_path0_deadline import estimate_path0_read

r = estimate_path0_read(
    kv_block_count=10,          # 10 block = 10 条 I/O
    deadline_ms=0.5,
    queued_io_count=100,        # 前方 100 条 176 KiB 请求
    bandwidth_gib_s=40,
    active_remaining_bytes=0,
)
print(r["required_io_count"])  # 10
print(r["estimated_read_ms"])  # 0.461578369140625
print(r["meets_deadline"])     # True
print(r["early_by_ms"])        # 0.038421630859375
print(r["late_by_ms"])         # 0
```

命令行同样输出 JSON：

```
python glm_path0_deadline.py \
  --kv-block-count 10 --deadline-ms 0.5 \
  --queued-io-count 100 --bandwidth-gib-s 40
```

参数和命令行只使用 GiB/s。若手中带宽为十进制 GB/s，请先换算： $B_{\text{GiB/s}} = B_{\text{GB/s}} \times 10^9 / 2^{30}$ 。例如 40 GB/s 约为 37.252903 GiB/s，不等于 40 GiB/s。

## 模型没有模拟什么？

- **无需额外假定 4 KiB 或 IOPS 来运行这个纯带宽公式。**完整 I/O 的 176 KiB 已经确定；40 GiB/s 下每条服务约 4.196167 微秒。这不是实机时延保证。
- **全部进入 Path0，不等于本批一次完整排好。**若提交时别的请求穿插，仅当前队列数不足以唯一预测本批完成时间，需要提交节奏和逐事件模拟。
- **不知道在途进度就不能给唯一精确值。**已知最多一条在途时，分别令  $A = 0$  和  $A = S$  可得本模型的上下界，相差最多一条服务时间；实机额外开销不在这个界内。
- **仅止于 SSD 完成，不含 HBM 接收、DRAM、固定命令开销或后台工作。**实机还需核实有效带宽、IOPS/延迟及并行度。NPU 纯计算时间不能从这些 I/O 输入推出。

测试运行：python -m unittest -q test\_glm\_path0\_deadline.py。仅保留核心计算与独立运行回归。**后文旧图与四卡反例仍使用原 trace，未因这次函数精简而重跑或改动数据。**

## 旧图确实自然错开了，但不是队列消失了

已核对本地 Git 中的 **commit** 9b7e332、用户指定原图，以及同提交 standalone 目录的认证 CSV、summary 和调用参数。它不是当前 4 卡反例的等条件对照：

| 条件        | 旧图                        | 当前反例                    |
|-----------|---------------------------|-------------------------|
| NPU / SSU | 32 / <b>11</b>            | 4 / <b>1</b>            |
| Path0     | 每个 SSU 各有自己的 Path0        | 唯一 SSU 的 Path0          |
| 总 SSD 容量  | 440 GiB/s                 | 40 GiB/s                |
| 每卡输入      | 全部 48K、NQL 512            | 四卡异构，见下一页               |
| 每层读取      | 每卡 65.3125 MiB，分散到 11 SSU | 每卡全部进入同一 SSU            |
| 每层计算      | 全部 5.044310 ms            | 0.582149 至 29.856563 ms |
| 请求 / 层数   | 每卡一个请求，3 层                | 持续 backlog，每请求 8 层      |
| 跨请求预取     | 关闭，也没有后续请求                | 开启                      |

两者都在**当前层开始计算时预取下一层**。旧图的 L0 在同一时刻发起，但冷读完成时间不同，因而 L0 计算开始、L1 预取发起自然分散：

| 读取层 | 32 卡发起时刻范围 (ms)      | 范围宽度 (ms) |
|-----|----------------------|-----------|
| L0  | 0 → 0                | 0         |
| L1  | 4.640721 → 5.081519  | 0.440798  |
| L2  | 9.685031 → 10.158881 | 0.473850  |

这是实实在在的错开。但 32 卡仍在不足 0.5 ms 内发起一轮读取，不是均匀分布在整个 5.044 ms 周期中。旧图蓝/红读取条从发起到 **HBM 就绪**，包含等待，并不是 SSD 持续服务条。

直接看旧图 NPU1：L1 从 **5.073127** 发起，到 **10.016211** 就绪，共 **4.943085 ms**，仍然很长，但小于其 **5.044310 ms** 计算预算，所以不 stall。L2 同样需要 **4.862840 ms**，也能隐藏。

甚至旧图也没有完全消除 stall：最忙的 SSD8 每层收到 1210 条 176 KiB 命令，服务需 **5.077362 ms**。减去计算预算，差 **0.033052 ms**，恰好等于 NPU17 在 L1、L2 各自剩下的 stall。

因此，正确结论是：**旧图错开后，多数读取能赶上各自 deadline；不是“错开后就不需要排队”。**

## 当前为什么还会反复受阻：谁的等待改变了时刻？

下面所有“时刻”与当前教程图 4 使用同一个零点：绝对仿真时间 **2430.127253 ms**，即中间 1 秒窗口的起点。L2 是从零编号的第 3 层；request\_id 只是请求标识，不是时刻或 I/O 条数。

先记住四张卡每层的任务。 $V$  是要读的 KV 数据量， $C$  是当前层可用来隐藏下一层 I/O 的计算时间：

| NPU | $V$ (MiB)  | $C$ (ms)  | 不排队的纯 SSD 服务 (ms) |
|-----|------------|-----------|-------------------|
| 0   | 1.148071   | 0.582149  | 0.028029          |
| 1   | 263.505859 | 12.390165 | 6.433249          |
| 2   | 262.796875 | 29.856563 | 6.415939          |
| 3   | 262.796875 | 29.856563 | 6.415939          |

## NPU2/3：排队了，但没有停算，因此不会继续被推开

| SSU 侧事件，单位 ms     | NPU2 / 请求 200010 | NPU3 / 请求 300010 |
|-------------------|------------------|------------------|
| 发起 L2 读取，同时开始算 L1 | 7.561568         | 7.557372         |
| L2 最后 SSD 完成      | 20.417279        | 20.241040        |
| L1 计算结束，发起 L3 读取  | 37.418131        | 37.413935        |
| 相邻两次读取发起间隔        | <b>29.856563</b> | <b>29.856563</b> |

NPU2 的 L2 在 SSD 侧耗时 12.855712 ms，HBM 就绪时间是 20.420636 ms；仍早于 37.418131 ms。它读取时在算 L1，**SSD 排队没有让它停下来**。NPU3 同理。两卡都继续按  $C = 29.856563$  ms 发起，约 4.196 微秒的相位差被保留。

严格地说，设  $r_k$  是第  $k$  次预取发起时刻， $L_k$  是这次预取从发起到 HBM 就绪的耗时；当前层计算时间固定为  $C$ ，没有其他调度空档，则：

$$r_{k+1} - r_k = \max(C, L_k) = C + \max(0, L_k - C).$$

右侧第二项才是**暴露出来的 stall**。不是只要 SSD 有排队，下一层就一定继续被推迟。对于这两张卡， $L_k < C$ ，所以直接得到  $P = C$ ；不必用数据量除以盘带宽去算周期。

## NPU0/1 确实被推迟，但新位置仍可能落在旧队列后

| 当前 trace 的事件        | NPU0 / 请求 111 / L7 | NPU1 / 请求 100020 / L3 |
|---------------------|--------------------|-----------------------|
| 本层读取发起时刻 (ms)       | 10.130560          | 9.387382              |
| 希望读完的 deadline (ms) | 10.712709          | 21.777547             |
| 第一条 SSD 服务开始 (ms)   | 26.850528          | 20.417279             |
| 最后一条 SSD 完成 (ms)    | 26.878557          | 26.850528             |
| 发起到最后 SSD 完成 (ms)   | <b>16.747997</b>   | <b>17.463146</b>      |
| 本卡计算预算 (ms)         | <b>0.582149</b>    | <b>12.390165</b>      |

例如 NPU0 已经因为前一次等待，把 L7 读取推迟到了 10.130560 ms；它与 NPU2/3 的 7.56 ms 发起**已经错开约 2.57 ms**。但 10.13 ms 不是空队列时刻。它仍须等到 26.850528 ms 才开始 SSD 服务。

其自身数据只有 1.148071 MiB、服务只需 0.028029 ms，真正长的是前方等待 **16.719968 ms**。读完并到 HBM 后，NPU0 的 L7 实际开始计算于 **26.881344 ms**，故暴露 stall 为：

$$26.881344 - 10.712709 = 16.168635 \text{ ms (显示值有舍入) .}$$

它并不是整个周期都在等；稍后有一段连续计算。原教程图 6 的“计算量 + 等待量”拼接条是**总量账本，不是按真实时刻排列的状态图**，不能据此说后半段始终等待。

### 你原来推理中少了哪个条件？

“等待会让下一层错开”对发生 stall 的卡成立；“**错开以后一定落入足够空的读取窗口**”并不随之成立。判断条件仍然是：

$$\text{前方剩余服务时间} + \text{自己的服务时间} \leq \text{本卡剩余预算}.$$

以 NPU0 为例，即使暂时忽略 HBM，它也只能容忍约 **22.696772 MiB** 的前方工作量，约 132 条完整 176 KiB 命令；NPU2 或 NPU3 的一次读取却各有 **1529 条**。仅从大流发起时间旁边移动几毫秒，不保证进入满足这个短 deadline 的位置。

因此，应把之前的表述修正为：**此例不是“所有卡都无法错开”，而是“两张不 stall 的大流保留近似同步的周期，其他卡虽然错开了，仍反复遇到超出自身短预算的排队”**。初始错位、队列是否存在、deadline 是否满足，是三个不同的问题。

## 对照实验、结论边界与复现

新增本地控制只改变 NPU3 的启动偏移，其他三卡保持原到达时刻。读取量、计算时间、单 SSU/Path0、每请求 8 层、跨请求预取全部不变，继续只测稳态中间 **1000 ms**。原例  $P = 29.856563$  ms。

| 启动偏移   | 稳态相位差 (ms) | 平均利用率      | NPU0       | NPU1       |
|--------|------------|------------|------------|------------|
| 原例 0   | 0.004196   | 76.226098% | 21.655961% | 83.248430% |
| $P/4$  | 6.461278   | 78.317299% | 15.152568% | 98.116628% |
| $P/2$  | 10.518068  | 78.439451% | 14.437307% | 99.320498% |
| $3P/4$ | 10.490039  | 78.439451% | 14.437307% | 99.320498% |

启动偏移量分别为 7.464141、14.928281、22.392422 ms。表中相位差则来自每例窗口内 29-30 对实际 L1-L7 发起事件：取以  $P$  为模数的最短圆周距离，不把启动偏移直接当稳态结果。所有例子 NPU2/3 仍为 100%。每例有两次模拟，观察器前后完整 summary 哈希相同。

**两张大卡真正错开了，平均值仍低于 80%；近乎同步不是低利用率的必要条件。**平均值有所改善，但 NPU0 反而更差、NPU1 更好，不能称为四卡都改善。也不能把三个点推广成“任何错位都无效”或全局最小值证明。

为什么单纯错开仍不保证够用？即使只有一张大卡的一次 262.796875 MiB 读取，其纯 SSD 服务也要 6.415939 ms，已远大于 NPU0 的 0.582149 ms 预算。**消除两张大卡的同时发起，不等于消除一张大卡留下的长队列。**新实验没有采集物理 I/O 明细，不能据此断言 NPU0 恶化的逐命令原因；“阻塞时段分散、连续好窗口减少”只能作为待检验解释。

原例四卡名义  $\sum V/C = 39.886158$  GiB/s，不代表 SSD 在整秒内一直满载：实测 SSD 忙碌比例为 **86.910452%**。停算会降低后续发起速率；有空闲时间与短 deadline 经常被错过并不矛盾。

旧图是单请求 3 层观察，不能凭它外推任意连续输入的长期利用率；当前输入也是构造反例，不是现实发生概率。特别是 NPU0 的 1K 计算画像用了数据表外推，而非 GLM5.1 实机测量。本轮无需远程机器；没有使用 SSH 或改动远程环境。

## 可复现文件

- 独立估算器：`glm_path0_deadline.py`。
- 单文件测试：`test_glm_path0_deadline.py`，仅保留核心计算和独立运行回归。
- 旧新 trace 审计：`audit_path0_staggering.py`。
- 审计结果：`path0_staggering_audit.json`，含原始输入 SHA256、逐层时间和数据量检查。
- 启动偏移实验：`run_baseline_path0_phase_probe.py`。
- 控制结果：`path0_phase_probe.json`，含参数、测量窗口和稳态实际相位。

在项目根目录运行前述测试或审计脚本即可。启动偏移脚本会重新仿真，比审计慢；不覆盖原始 `result.json`、`trace` 或旧 PDF。

旧证据定位：`commit 9b7e332` 的以下目录：

`standalone/npu32_baseline_timelines/`

其中，`run_timelines.py` 保存配置；`results/` 下对应 `layer012` 子目录保存认证摘要与层事件 CSV。根目录用户指定 PNG 与 `standalone` 图的布局不同，但底层认证事件一致。

此次说明明确区分“有错位”“没有队列”和“满足预取预算”，并记录了纠正。原版教程保留，便于追溯。